

# **StreamReader и StreamWriter**

# Запись в файл и StreamWriter

```
string path = "note.txt";  
StreamWriter sw1 = new StreamWriter(path);  
  
StreamWriter sw2 = new StreamWriter(path, true);  
  
StreamWriter sw3 = new StreamWriter(path, true, Encoding.UTF8);
```

Для записи в текстовый файл используется класс `StreamWriter`. Некоторые из его конструкторов, которые могут применяться для создания объекта `StreamWriter`:

**`StreamWriter(string path)`:** через параметр `path` передается путь к файлу, который будет связан с потоком

**`StreamWriter(string path, bool append)`:** параметр `append` указывает, надо ли добавлять в конец файла данные или же перезаписывать файл. Если равно `true`, то новые данные добавляются в конец файла. Если равно `false`, то файл перезаписывается заново

**`StreamWriter(string path, bool append, System.Text.Encoding encoding)`:** параметр `encoding` указывает на кодировку, которая будет применяться при записи

# Методы StreamWriter - Close

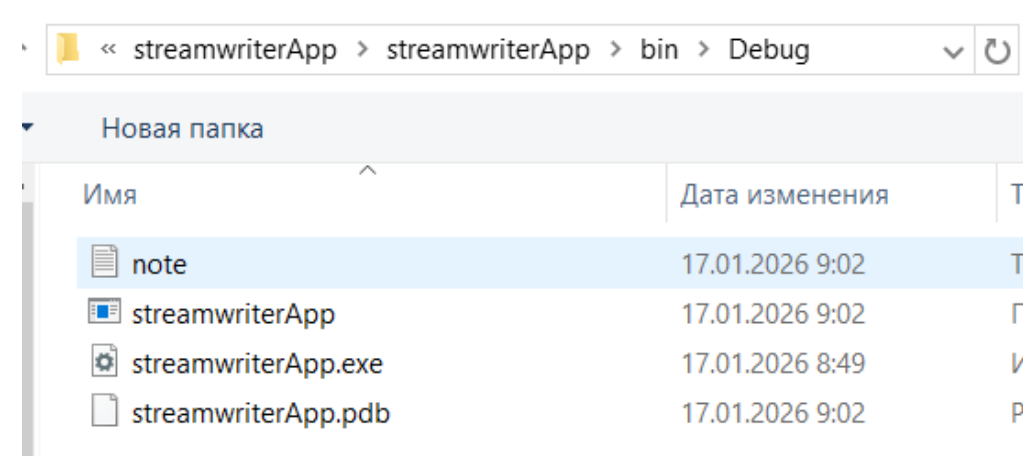
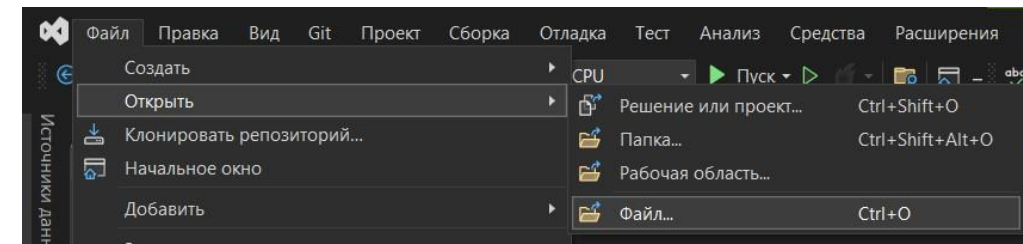
int Close(): закрывает записываемый файл и освобождает все ресурсы

Для того чтобы не закрывать файл, можно использовать using ()

```
string path = "note.txt";  
StreamWriter sw1 = new StreamWriter(path);  
sw1.Close();
```

```
string path = "note.txt";  
using (StreamWriter sw1 = new StreamWriter(path)) { }  
if (File.Exists(path))  
{  
    Console.WriteLine("Файл создан");  
}
```

Создание файла note.txt

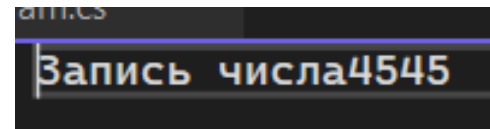


# Методы StreamWriter-Write

`void Write(string value)`: записывает в файл данные простейших типов, как `int`, `double`, `char`, `string` и т.д. Соответственно имеет ряд перегруженных версий для записи данных элементарных типов, например, `Write(char value)`, `Write(int value)`, `Write(double value)` и т.д.

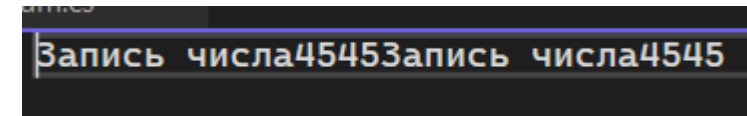
`Task WriteAsync(string value)`: асинхронная версия метода `Write`. Обратите внимание, что асинхронные версии есть не для всех перегрузок метода `Write`.

```
string path = "note.txt";
using (StreamWriter sw1 = new StreamWriter(path))
{
    sw1.Write("Запись числа");
    sw1.Write(4545);
}
```

A screenshot of a text editor showing a file named 'note.txt' with the text 'Запись числа4545'.

Полная перезапись

```
string path = "note.txt";
using (StreamWriter sw1 = new StreamWriter(path, true))
{
    sw1.Write("Запись числа");
    sw1.Write(4545);
}
```

A screenshot of a text editor showing a file named 'note.txt' with the text 'Запись числа4545Запись числа4545'.

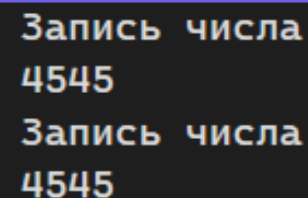
Дозапись перезапись

# Методы StreamWriter - WriteLine

`void WriteLine(string value)`: также записывает данные, только после записи добавляет в файл символ окончания строки

`Task WriteLineAsync(string value)`: асинхронная версия метода `WriteLine`

```
string path = "note.txt";  
using (StreamWriter sw1 = new StreamWriter(path, true))  
{  
    sw1.WriteLine("Запись числа");  
    sw1.WriteLine(4545);  
}
```



Запись числа  
4545

# Асинхронные методы

Для вызова асинхронного метода, он должен вызываться в методе с модификатором **async**, перед вызовом прописывается оператор **await**.

Основные преимущества

Неблокирующий UI: В десктопных/веб-приложениях поток интерфейса не "зависает" при записи файлов или запросах к сети.

Экономия потоков: Вместо создания потока на каждую задачу используется один поток для множества операций.

Масштабируемость: Сервер может обрабатывать тысячи запросов одновременно без роста потребления памяти.

Ссылка: 1

```
static async void MainWriteLine()
{
    string path = "example_async.txt";

    using (StreamWriter writer = new StreamWriter(path, true))
    {
        // Асинхронная запись строки в файл
        await writer.WriteLineAsync("Первая строка");
        await writer.WriteLineAsync("Вторая строка");
    }

    Console.WriteLine("Асинхронная запись завершена.");
}
```

Ссылка: 0

```
static void Main(string[] args)
{
    MainWriteLine();
}
```



# Методы StreamWriter - Flush

`void Flush()`: записывает в файл оставшиеся в буфере данные и очищает буфер. То есть весь текст, который был записан до `Flush` запишется в файл, остальной текст ниже запишется только после `Close` или окончания `using`

`Task FlushAsync()`: асинхронная версия метода `Flush`

```
string path = "log.txt";

// Открываем файл для дозаписи
using (StreamWriter writer = new StreamWriter(path, append: true, Encoding.UTF8)
{
    writer.WriteLine("Старт программы: " + DateTime.Now);

    // Явно сбрасываем буфер на диск
    writer.Flush();
    Console.WriteLine("Произошла запись в файл, без его закрытия, " +
        "нажмите Enter, чтобы продолжить");
    Console.ReadKey();

    writer.WriteLine("Ещё одна строка лога");
    writer.WriteLine("Конец программы: " + DateTime.Now);
}

Console.WriteLine("Запись завершена.");
```

```
Старт программы: 17.01.2026 9:27:58
Ещё одна строка лога
Конец программы: 17.01.2026 9:29:29
```

Без `Flush` и `ReadKey`

```
Старт программы: 17.01.2026 9:33:18
Ещё одна строка лога
Конец программы: 17.01.2026 9:33:18
```

# Чтение из файла и StreamReader

```
string path = "note.txt";  
StreamReader swr1 = new StreamReader(path);  
StreamReader swr2 = new StreamReader(path, Encoding.UTF8);
```

Класс **StreamReader** позволяет нам легко считывать весь текст или отдельные строки из текстового файла.

Некоторые из конструкторов класса **StreamReader**:

**StreamReader(string path)**: через параметр `path` передается путь к считываемому файлу

**StreamReader(string path, System.Text.Encoding encoding)**: параметр `encoding` задает кодировку для чтения файла



# Методы StreamReader - Read

**int Read():** считывает и возвращает следующий символ в численном представлении. Имеет перегруженную версию: **Read(char[] array, int index, int count)**, где array - массив, куда считываются символы, index - индекс в массиве array, начиная с которого записываются считываемые символы, и count - максимальное количество считываемых символов

**Task<int> ReadAsync():** асинхронная версия метода Read

```
string path = "note.txt";

using (StreamReader reader = new StreamReader(path))
{
    int symbolCode;
    // Читаем посимвольно до конца файла
    while ((symbolCode = reader.Read()) != -1)
    {
        char symbol = (char)symbolCode;
        Console.Write(symbol);
    }
}
```

Консоль отладки Microsoft Visual Studio

// Дорогой дневник...

```
string path = "note.txt";

using (StreamReader reader = new StreamReader(path))
{
    char[] array = new char[new FileInfo(path).Length];
    reader.Read(array, 0, array.Length);
    Console.WriteLine(String.Join("", array));
}
```

Консоль отладки Microsoft Visual Studio

Дорогой дневник...

# Методы StreamReader

int Peek(): возвращает следующий доступный символ, если символов больше нет, то возвращает -1

```
string path = "note.txt";
File.WriteAllText(path, "ABC123");

using (var reader = new StreamReader(path))
{
    // Peek() = 65 ('A'), позиция не меняется
    Console.WriteLine($"Следующий: {(char)reader.Peek()}");

    // Read() = 65 ('A'), позиция сдвигается
    Console.WriteLine($"Прочитано: {reader.Read()} -> '{(char)65}'");

    // Peek() = 66 ('B')
    Console.WriteLine($"Следующий: {(char)reader.Peek()}");
}
```

 Консоль отладки Microsoft Visual Studio

```
// Следующий: A
// Прочитано: 65 -> 'A'
// Следующий: B
//
```

# Методы StreamReader - ReadLine

string ReadLine(): считывает одну строку в файле, возвращает строку

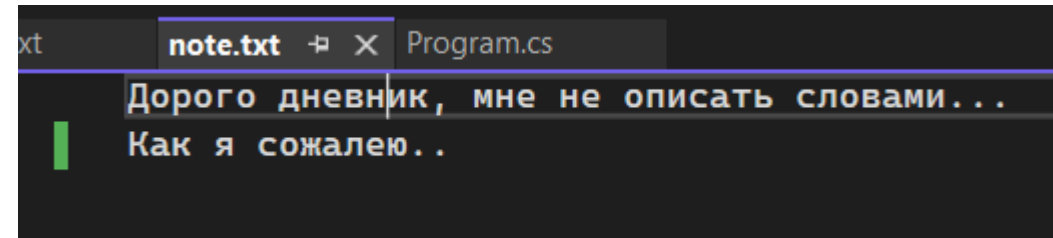
string ReadLineAsync(): асинхронная версия метода ReadLine

```
string path = "note.txt";

using (StreamReader reader = new StreamReader(path))
{
    string line;
    while ((line = reader.ReadLine()) != null)
    {
        Console.WriteLine(line);
    }
}
```

Консоль отладки Microsoft Visual Studio

Дорого дневник, мне не описать словами...  
Как я сожалею..

A screenshot of a text editor window with two tabs: 'note.txt' and 'Program.cs'. The 'note.txt' tab is active, showing the text: 'Дорого дневник, мне не описать словами...' followed by 'Как я сожалею..' on the next line. A green cursor is positioned at the end of the first line.

note.txt Program.cs

Дорого дневник, мне не описать словами...  
Как я сожалею..

```
string path = "note.txt";

using (StreamReader reader = new StreamReader(path))
{
    string line;
    List<string> lst = new List<string>();
    while ((line = reader.ReadLine()) != null)
    {
        lst.Add(line);
    }
}
```

# Методы StreamReader - ReadToEnd

string ReadToEnd(): считывает весь текст из файла

string ReadToEndAsync(): асинхронная версия метода ReadToEnd

```
string path = "note.txt";

using (StreamReader reader = new StreamReader(path))
{
    string text = reader.ReadToEnd();
    Console.WriteLine(text);
}
```

 Консоль отладки Microsoft Visual Studio

```
Дорого дневник, мне не описать словами...
Как я сожалею..
```

# Методы StreamReader - Close

void Close(): закрывает считываемый файл и освобождает все ресурсы

```
string path = "note.txt";
StreamReader swr1 = new StreamReader(path);
string text_file = swr1.ReadToEnd();
swr1.Close();
Console.WriteLine(text_file);

using (StreamReader swr2 = new StreamReader(path, Encoding.UTF8))
{
    string text_file_2 = swr2.ReadToEnd();
    Console.WriteLine(text_file_2);
}
```

 Выбрать Консоль отладки Microsoft Visual Studio

Дорогой дневник...  
Дорогой дневник...

# Пример с изменением текста

- Создайте файл `task.txt`, запишите в него пару задач, и выведите в консоль.
- Добавьте в конец новую задачу, выведите результат в консоль.
- Добавьте в начало строку «Задачи на сегодня:», выведите результат в консоль.
- Замените слово «задача» в тексте файла на слово «цель»,
- выведите результат в консоль.
- *Алгоритм решения для изменения: читаем файл и записываем в список строк или строку, изменяем переменную по заданию, перезаписываем файл полностью.*



# Задание с StreamReader и StreamWriter

**Цель:** Создать консольное приложение с использованием классов StreamReader и StreamWriter для выполнения операций с текстовыми файлами. Меню должно работать бесконечно, все действия должны быть прописаны в *отдельных методах*.

## **Обязательные функции меню:**

- Выбор/создание файла - ввод пути к текстовому файлу с проверкой существования
- Чтение посимвольно - чтение символов с указанием количества (до N-го символа)
- Чтение построчно - отображение конкретной строки по номеру (1-based)
- Чтение целиком - вывод всего содержимого файла
- Запись строки в конец - добавление новой строки в конец файла
- Вставка строки по номеру - вставка строки перед указанной позицией со смещением последующих строк(используйте List<string> )
- Удаление строки - удаление строки по заданному номеру
- Полная перезапись - замена всего содержимого файла новым текстом
- Выход из программы.